

MCP PYTHON SDK V2 · MIGRATION & INTEROPERABILITY

Upgrading your MCP server to v2 — *without breaking the agents.*

A practical migration guide for teams running production MCP servers. The v2 SDK supports the 2026-07-28 spec — it is a **major rework, not a safe dependency bump**.

Scope: written for teams running **Python** MCP servers on **Streamable HTTP**, especially mounted behind FastAPI. The migration protocol and interoperability test card apply to any language; the code specifics are Python + FastAPI. `studio` / local servers don't touch the transport half.

Technical specifics below were reviewed against the official MCP Python SDK v2 migration guide and API reference. Confirm against the current SDK before your own production rollout.

THE SHORT VERSION

Keep production pinned to a working v1 release while you port and test v2 on a separate branch and endpoint. Installing `mcp ≥ 2` over v1 code will often *install* cleanly and then fail at runtime — on imports, constructor arguments, routes, startup, or identity handling. Migrate deliberately; canary; keep a v1 rollback until the observation window closes.

1 What changed in SDK v2

The high-level server model is still recognizable — but construction, transport, context, and lifecycle APIs moved. These are the changes most likely to affect a FastAPI-mounted MCP server. ✓ marks items documented in the official v2 migration guide or API reference.

V1 PATTERN	V2 MIGRATION	OFFICIAL DOCS
FastMCP	Renamed to <code>MCPServer</code> .	✓
<code>mcp.server.fastmcp</code>	Module removed. Import from <code>mcp.server.mcpserver</code> .	✓
Transport in constructor	Pass <code>host</code> , <code>streamable_http_path</code> , <code>stateless_http</code> , <code>json_response</code> to <code>streamable_http_app()</code> (or <code>run()</code>).	✓
Reading HTTP headers	Use <code>ctx.headers</code> for transport-provided request headers. It is <code>None</code> on transports such as stdio. <code>ctx.request_context</code> still exists for request metadata and lifespan context.	✓
Implicit server version	Pass an explicit <code>version=</code> .	✓
Session-manager access	Public <code>mcp.session_manager</code> property — created lazily , only after <code>streamable_http_app()</code> is called.	✓
Per-request lifespan	Streamable-HTTP lifespan is entered once at session-manager start, not per request.	—
Auto <code>.env</code> loading	Load configuration in the application itself.	—

What stays familiar: stateless Streamable HTTP is still supported and carries no state between requests; tools still use `@mcp.tool()`; a param annotated `ctx: Context` is still how a tool receives context. Authentication concepts remain, but your middleware and identity propagation still need deliberate testing.

REPRESENTATIVE PORT — FASTAPI-MOUNTED SERVER

```
# Before (v1)
from mcp.server.fastmcp import FastMCP, Context
mcp = FastMCP("Conxt", stateless_http=True, json_response=True,
              streamable_http_path="/")
app.mount("/mcp", mcp.streamable_http_app())

# After (v2)
from mcp.server.mcpserver import MCPServer, Context
mcp = MCPServer("Conxt", version="1.2.0")
mcp_app = mcp.streamable_http_app(
    streamable_http_path="/", # REQUIRED when mounting at /mcp:
    stateless_http=True,     # v2 default is "/mcp" → would give /mcp/mcp
    json_response=True,
)
app.mount("/mcp", mcp_app)
```

FASTAPI-MOUNTED DETAILS TO TEST EXPLICITLY

Four details documented by the official v2 guidance can bite a FastAPI-mounted server: **(1)**

`streamable_http_app()` 's default path is `/mcp` — mount at `/mcp` and you get `/mcp/mcp` unless you pass `streamable_http_path="/"`. **(2)** Read transport-provided HTTP headers through

`ctx.headers` ; do not depend on the transport-specific object behind

`ctx.request_context.request`. **(3)** `mcp.session_manager` raises `RuntimeError` if touched before `streamable_http_app()` runs — build the mounted app at module level so the manager exists by the time the host lifespan starts. **(4)** The Streamable-HTTP request-body limit defaults to **4 MiB** (`4194304`).

COMMON MISTAKE

Do not change `mcp[cli]=1.x` to `mcp ≥ 2` and deploy the same code. The package installs; the imports, constructor arguments, routes, startup, and identity handling do not.

2 Safe migration protocol

A staged protocol so beta feedback measures interoperability — not an uncontrolled production upgrade.

1 Freeze the baseline.

Pin the working v1 dependency; capture its tool list, schemas, auth behavior, latency, and known clients.

2 Add regression tests first.

Cover initialize, `tools/list`, representative tool calls, invalid input, unauthorized access, startup, shutdown, and concurrent requests.

3 Create a v2 branch.

Change imports and construction; move transport settings to the app factory.

4 Review lifecycle assumptions.

Keep process-wide pools and caches in lifespan; move per-request acquisition out of it.

5 Review identity propagation.

Verify middleware-authenticated identity reaches tools through injected context. Preserve non-JWT API-key paths intentionally.

6 Deploy staging.

Separate service or hostname, separate logs, immediate rollback path.

7 Run interoperability tests.

Direct clients and gateways against the same tool set; compare schemas, error shapes, headers, and routes.

8 Canary and cut over.

Limited traffic to v2, observe, promote only after parity, retain v1 until the window closes.

Route & transport checks

Confirm both `/mcp` and `/mcp/` behave, with no nested `/mcp/mcp`. Verify `Content-Type`, `Accept`, and the new routing headers (`Mcp-Method`, `Mcp-Name`). Exercise stateless mode under concurrency — never assume request state survives. Test the 4 MiB body limit. Validate DNS-rebinding / Host protections behind FastAPI, Railway, a proxy, or a gateway.

Authentication & governance checks

Reject missing, expired, malformed, wrong-issuer, and wrong-audience tokens. Keep permanent-API-key and OAuth/JWT paths distinct and observable. Pass verified identity from middleware — never trust a caller-provided user ID for shared writes. Enforce scopes at the *action boundary*, not only in the prompt. Log the tool, actor, tenant, authorization result, downstream action, and recovery — never secrets.

3 Interoperability test card

For every real workflow. A successful tool call is not enough — the beta should demonstrate safe execution, traceability, and recovery across server boundaries.

AREA	PASS EVIDENCE	FAILURE TO WATCH
Discovery	Client initializes; expected server identity and tool list.	Missing tools, changed names, schema drift.
Schema	Arguments and structured outputs validate consistently.	Optional/required fields differ across clients.
Authentication	Correct principal, tenant, audience, scopes reach the tool.	Anonymous fallback or caller-supplied identity trusted.
Authorization	Denied actions fail before any downstream mutation.	Prompt-level guardrail is the only control.
Execution	Integration layer invokes the business endpoint exactly once.	Duplicate, partial, or wrong-tenant action.
Observability	One trace links intent, MCP call, approval, API action, result.	Logs cannot reconstruct what happened.
Recovery	Timeout/retry is idempotent; rollback is defined.	Retry duplicates a real-world action.
Compatibility	Claude, Codex, Cursor, VS Code, gateways agree on behavior.	One client silently reads the schema differently.

Recommended server boundaries

Keep *context & governance* and *universal execution* as separate MCP responsibilities — clearer permissions, and each server evolves or rolls back independently.

Agent / LLM	Interpret the request, gather context, propose a plan, request tools.
Context / governance MCP	Approved memory, workflow rules, decisions, guardrails, required approvals.
Execution MCP / gateway	Governed execution across business apps through one integration layer.
Business system	System of record and final source of action state.

THE QUESTION THE BETA SHOULD ANSWER

Can we reconstruct *who* asked for an action, *which* context and policy allowed it, *what* the execution layer performed, *whether* the business system committed it, and *how* we recovered if any step failed?

4 Release gate

Do not promote v2 until all are true.

- ☐ The SDK is constrained intentionally (v1 pinned, or v2 bounded below the next major).
- ☐ All server imports and constructor arguments match the v2 API.
- ☐ The final mounted MCP route is confirmed from outside the deployment network.
- ☐ Lifespan starts and stops cleanly; shared resources initialize exactly once.
- ☐ Every production tool appears with the expected schema and compatible output.
- ☐ JWT/OAuth and permanent-API-key paths both pass positive and negative tests.
- ☐ Agent, execution layer, and downstream API share a trace/correlation ID.
- ☐ Retries are idempotent or guarded by a deduplication key.
- ☐ A known-good v1 rollback remains available.

Official references

Python SDK · github.com/modelcontextprotocol/python-sdk

v2 migration guide · py.sdk.modelcontextprotocol.io/migration

Specification (2026-07-28) · modelcontextprotocol.io/specification/2026-07-28

Community implementation aid—not an official MCP publication. Confirm against official sources before production use.